

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

SAMEER TONDUR (VTU25541)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "SMART CAMPUS EVENT MANAGEMENT SYSTEM" by SAMEER TONDUR (VTU25541) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.S.Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr.M.Sumithra

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

T. Sameer

Date:06/05/2026

ABSTRACT

The Smart Campus Event Management System is a web-based application designed to streamline the creation, management, and registration of campus events in universities and colleges. It provides a centralized platform where students can securely register, explore upcoming events, apply advanced search and filtering options, and manage their event participation. Administrators are provided with dedicated tools to create, update, and delete events, monitor registrations, and analyze participation through real-time statistics. The system incorporates modern technologies such as a React-based frontend and a Spring Boot backend with RESTful APIs, ensuring efficient communication and scalability. By replacing manual event management processes, the system minimizes errors, avoids overbooking through capacity constraints, saves time, and enhances overall user experience and operational efficiency.

Keywords: Smart Campus, Event Management System, React.js, Spring Boot, REST API, User Authentication, Event Registration, H2 Database, Axios, Single Page Application, CRUD Operations, Web Application, Automation

LIST OF FIGURES

4.1	System Architecture	10
4.2	Data Flow Diagram	11
5.1	Landing Page Interface	15
5.2	Event Registration page	16

LIST OF TABLES

2.1	Comparative Analysis of Market Alternatives	5
7.1	Mapping of Project to Complex Engineering Problem (CEP)	20
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	21
7.3	SDG Mapping for Smart Campus Event Management System	22

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
Auth	Authentication
CRUD	Create, Read, Update, Delete
DB	Database
HTTP	HyperText Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
MVC	Model View Controller
REST	Representational State Transfer
UI	User Interface

TABLE OF CONTENTS

ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6
3.1.1 Environment Setup	6
3.1.2 Structure of the Project	6
3.1.3 Execution Procedure	7
3.2 Backend Implementation	7
3.2.1 Environment Setup	7

3.2.2	Structure of the Project	8
3.2.3	Execution Procedure	8
3.3	Database Implementation	8
3.3.1	Database Configuration	8
3.3.2	Schema Structure	9
3.3.3	Tables created	9
4	METHODOLOGIES	10
4.1	Project Architecture	10
4.2	DataFlow Diagram	11
4.3	System Modules	11
4.3.1	Module 1: User Management	12
4.3.2	Module 2: Event Management	12
4.3.3	Module 3: Event Registration	12
4.4	Advantages and Limitations of the Project	12
4.4.1	Advantages	13
4.4.2	Limitations	13
5	RESULTS AND DISCUSSIONS	14
5.1	System Implementation Overview	14
5.2	System Interface Visualizations	15
5.3	Discussion	16
6	CONCLUSION AND FUTURE ENHANCEMENTS	17
6.1	Conclusion	17
6.2	Future Enhancements	18
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	19

7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	19
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	21
7.3	SDG Mapping (CEP Section)	22
8	SOURCE CODE	23
8.1	Source Code	23
	REFERENCES	25

Chapter 1

INTRODUCTION

1.1 Introduction

In the era of digital transformation, educational institutions are increasingly adopting smart solutions to streamline administrative and academic activities. One of the key areas that still relies on semi-manual processes is campus event management, which includes event creation, registration, and participation tracking. Traditional methods such as manual registrations, spreadsheets, and notice boards are time-consuming, error-prone, and inefficient when handling large numbers of students.

The Smart Campus Event Management System is developed as a modern web-based solution to overcome these challenges. It provides a centralized platform where students can explore, filter, and register for events, while administrators can efficiently manage event creation, monitor registrations, and analyze participation data.

The system leverages a full-stack architecture using modern technologies, ensuring scalability, responsiveness, and ease of use. It enhances transparency, reduces manual workload, prevents issues like overbooking through capacity constraints, and improves overall user experience for both students and administrators.

1.2 Aim of the Project

The primary aim of the Smart Campus Event Management System is to design and develop a user-friendly and efficient web-based platform for managing campus events and student participation. The primary objective of the system is to automate the event management process within educational institutions, thereby reducing manual effort and improving operational efficiency. The system provides a centralized platform where students can easily explore available events and register seamlessly.

It also enables administrators to efficiently create, update, and manage events through a structured interface. By incorporating capacity constraints, the system minimizes manual errors and prevents issues such as overbooking.

Overall, the system enhances efficiency, accessibility, and user experience by offering a reliable and user-friendly solution for managing campus events.

1.3 Scope of the Project

The Smart Campus Event Management System focuses on developing a web-based platform that enables efficient management of campus events and student participation within educational institutions. The system allows students to explore available events, apply advanced search and filtering based on criteria such as event name, date, and department, and register for events online while ensuring capacity constraints are maintained to prevent overbooking. It also provides administrators with tools to create, update, and delete events, as well as monitor all student registrations and analyze participation through basic statistics. The platform is designed using a modern full-stack architecture with a React-based frontend, a Spring Boot backend, and an H2 database for data storage. The system aims to reduce manual

effort, improve accuracy, and streamline event coordination, while being scalable for future enhancements such as notifications, analytics, and integration with other institutional systems.

1.4 Framework

The system framework consists of multiple layers including the user interface, application logic, and database. The frontend layer is developed using React.js to provide an interactive user experience. The backend layer is built using Spring Boot, which handles business logic and RESTful API communication. The database layer uses an H2 in-memory database to store event, student, and registration data efficiently.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several event management and registration applications are widely used in the market, such as Eventbrite, Meetup, Townscript, and Cvent. These platforms provide essential features including event creation, online registration, ticket management, and user-friendly interfaces that enhance overall user experience.

In addition to these, platforms like Whova and Bizzabo offer advanced functionalities such as attendee engagement tools, event analytics, networking features, and automated communication. These features help organizers efficiently manage large-scale events and track participant interaction.

Furthermore, institutional portals and basic registration systems are also used within universities for managing internal events. These systems provide simple functionalities such as event listing, manual registration, and limited tracking mechanisms. However, they often lack scalability, modern interfaces, and advanced filtering capabilities.

Most existing applications are primarily designed for large-scale public events such as conferences, exhibitions, and commercial gatherings. They commonly include features like ticket booking, payment integration, analytics dashboards, and

promotional tools that support large user bases.

However, these platforms often lack specific support for academic or campus-level event management, where requirements such as department-based filtering, restricted access, capacity management, and simplified workflows are essential. This gap highlights the need for a dedicated system tailored for campus environments.

Feature	BookMyShow	Eventbrite	Townscript	Your Project
Event Creation	Yes	Yes	Yes	Yes
Ticket Booking	Yes	Yes	Yes	Yes
Internal Department Events	No	No	No	Yes
User Registration/Login	Yes	Yes	Yes	Yes
Online Payment Integration	Yes	Yes	Yes	No / Planned
QR Code / E-Ticket	Yes	Yes	Yes	Yes
Seat Selection	Yes	Limited	Limited	Yes
Event Analytics	Yes	Yes	Yes	Basic
Direct Communication (Admin-User)	No	Limited	No	Yes

Table 2.1: Comparative Analysis of Market Alternatives

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is built using modern web technologies such as React.js and Vite. Node.js is used as the runtime environment for managing dependencies and running the development server. Tools like Visual Studio Code are used for coding, while web browsers are used for testing and debugging. Additional libraries such as Axios are used for API communication, and React Router DOM is used for navigation within the application.

3.1.2 Structure of the Project

The frontend follows a modular and component-based architecture to ensure scalability, maintainability, and efficient development.

The public directory contains static files such as the main HTML file and images used across the application. The source directory is organized into multiple folders, including components, pages, and services.

Reusable UI components such as navigation bars, event cards, and forms are stored in the components folder. The pages folder contains the main screens of the appli-

cation, including the landing page, student dashboard, admin dashboard, login, and registration pages. The services folder manages API calls using Axios, ensuring smooth communication with the backend server.

This structured approach helps in maintaining clean code and enables easy expansion of features.

3.1.3 Execution Procedure

The frontend application is executed by running the development server using Node.js. Once the server starts, the application is accessible through a web browser. Users can interact with the interface to browse events, apply filters, register for events, and access dashboards. Developers can test and debug features in real time during execution.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Spring Boot, which provides a robust framework for building RESTful APIs and handling business logic. Java Development Kit (JDK) is required for development, and Maven is used for dependency management and project build lifecycle.

Spring Boot simplifies configuration and deployment through its embedded server. Tools such as Postman are used for testing API endpoints, and Git is used for version control and collaboration.

3.2.2 Structure of the Project

The backend project follows a layered architecture consisting of controllers, services, repositories, and model classes.

The controller layer handles incoming HTTP requests and defines API endpoints. The service layer contains business logic such as event management, registration handling, and validation. The repository layer uses Spring Data JPA to interact with the database without writing complex SQL queries. The model layer defines entity classes such as Student, Event, and Registration.

This structured approach ensures separation of concerns, improves maintainability, and supports scalability.

3.2.3 Execution Procedure

The backend server is started using Maven commands or directly from an IDE. Once the server is running, it listens for client requests and processes them through defined REST APIs. The backend communicates with the database to store and retrieve data, and responses are sent back to the frontend in JSON format.

3.3 Database Implementation

3.3.1 Database Configuration

The system uses an H2 in-memory database for storing application data. It is configured within the Spring Boot application using properties files. The database automatically initializes during runtime and provides fast data access without requiring external setup.

3.3.2 Schema Structure

The database schema is designed using a relational model, organizing data into structured tables with proper relationships. Entities such as students, events, and registrations are connected using primary and foreign keys to maintain data integrity and consistency.

3.3.3 Tables created

The core functionalities of the system are supported by a well-structured relational database that ensures efficient data storage, retrieval, and management. The database is designed to maintain consistency, avoid redundancy, and support smooth interaction between different modules of the system.

The Student table plays a crucial role in managing user information. It stores essential details such as student ID, name, email, department, and password, which are used for authentication and authorization. This table ensures that only registered users can access the system and participate in events.

The Event table is responsible for maintaining all event-related information. It includes attributes such as event title, date, department, type, description, and maximum capacity. This enables administrators to create, update, and manage events efficiently while ensuring proper organization and scheduling.

The Registration table acts as a bridge between the Student and Event tables, representing the relationship between users and events. It stores details such as registration ID, student ID, and event ID, ensuring accurate tracking of participation. This table prevents duplicate registrations and helps manage event capacity effectively.

In addition to these primary tables, the system design supports data integrity through proper use of keys and constraints. Primary keys uniquely identify each record, while foreign keys maintain relationships between tables.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Smart Campus Event Management System follows a layered architecture consisting of frontend, backend, and database components. The frontend is responsible for handling user interactions and providing a dynamic interface for students and administrators. The backend manages business logic, processes requests, and handles communication between the frontend and the database. The database stores all essential data such as student details, event information, and registrations. This structured architecture ensures efficient data flow, improved security, and scalability of the system[Figure 4.1].

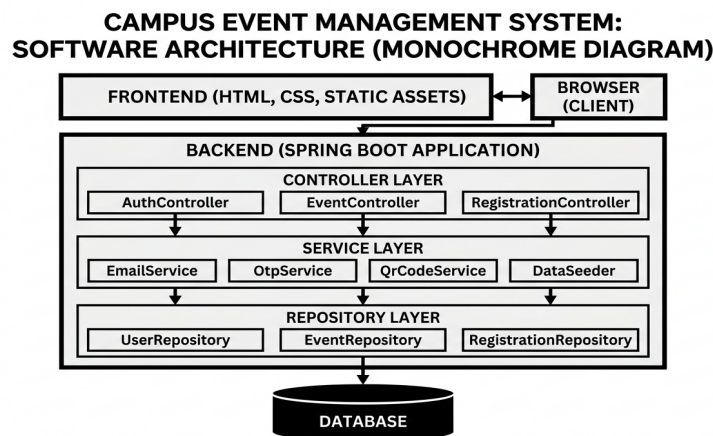


Figure 4.1: System Architecture

4.2 DataFlow Diagram

The Data Flow Diagram illustrates how data moves within the system. User inputs, such as login details or event registration requests, are sent from the frontend to the backend through API calls. The backend processes these requests, performs necessary validations such as checking event capacity, and interacts with the database to store or retrieve information. The processed data is then sent back to the frontend, which displays outputs such as event lists, registration confirmations, and user dashboards[Figure 4.2].

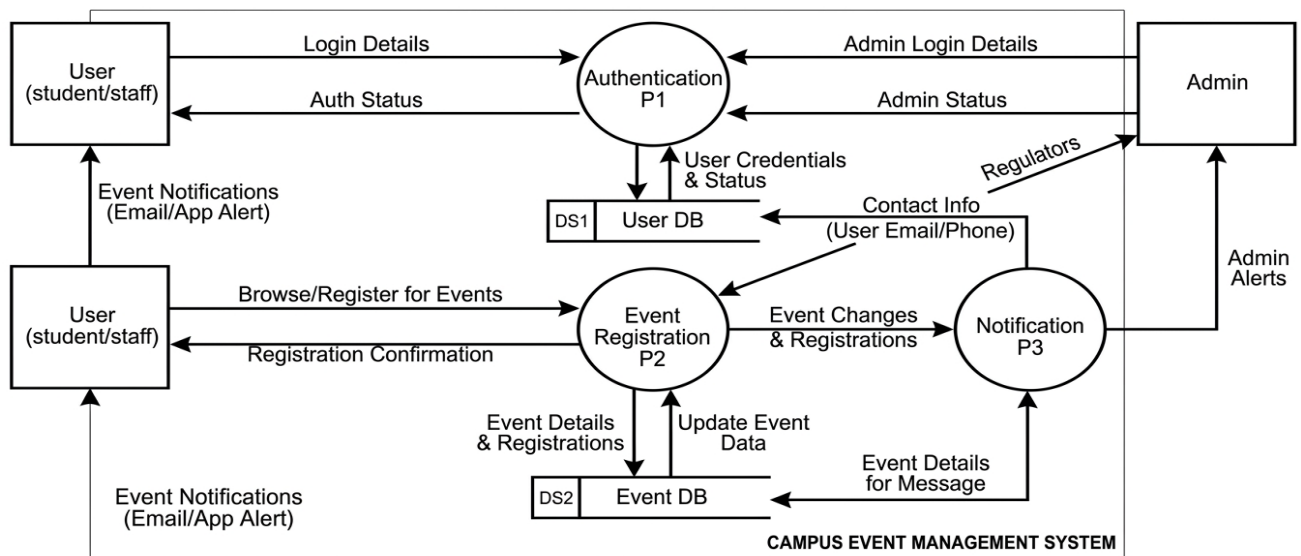


Figure 4.2: Data Flow Diagram

4.3 System Modules

The system is divided into key modules including User Management, Event Management, and Event Registration to ensure organized functionality. Each module handles specific operations such as authentication, event creation, and student registrations. This modular approach improves system efficiency, scalability, and ease of maintenance.

4.3.1 Module 1: User Management

This module handles student registration, login, and authentication processes. It ensures that only authorized users can access the system and perform actions such as event registration. The module securely stores user credentials and validates login information. It also manages user sessions and maintains data integrity.

4.3.2 Module 2: Event Management

This module is designed for administrators to manage events efficiently. It allows admins to create new events, update existing event details, and delete events when necessary. Information such as event title, date, department, type, description, and capacity is maintained. This module ensures proper organization and control over all campus activities.

4.3.3 Module 3: Event Registration

This module enables students to register for events easily. It allows users to view event details, check seat availability, and complete registration. The system validates each registration to prevent duplicate entries and ensures that the event capacity is not exceeded. Once registration is successful, confirmation details are provided to the user.

4.4 Advantages and Limitations of the Project

The Smart Campus Event Management System offers several advantages by automating and simplifying the event management process. It reduces manual workload, improves efficiency, and ensures accurate data handling. The system provides a user-friendly interface for both students and administrators, making event coordi-

nation seamless.

4.4.1 Advantages

The system offers several advantages that improve efficiency and usability. It saves time by automating event management and registration processes, thereby reducing manual effort. The system also minimizes human errors by preventing issues such as duplicate registrations and overbooking.

In addition, it provides easy management for administrators by enabling them to efficiently organize events, monitor activities, and track student participation in a structured manner.

4.4.2 Limitations

The system depends on internet connectivity for proper functioning, and performance may be affected in case of network issues. Additionally, initial development and setup require technical knowledge and time. Regular maintenance is necessary to ensure smooth operation and to handle potential system issues.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Implementation Overview

The Smart Campus Event Management System is designed as a comprehensive web-based platform integrating multiple modules for both students and administrators to efficiently manage campus events and participation.

- Landing Page Interface
- Authentication Module
- Student Dashboard
- Admin Dashboard
- Event Management Module
- Event Registration Module
- Advanced Search & Filtering
- My Registrations (Student Panel)
- Analytics Dashboard
- Notification System

5.2 System Interface Visualizations

The system interface is designed to provide a user-friendly and intuitive experience for all users. The application follows a modern web-based design with clear navigation and visually appealing components.

The landing page interface serves as the entry point of the system. It presents key features such as event discovery, registration, and user account management. The interface includes a navigation bar with options like Home, Browse Events, and My Events, allowing users to easily access different modules[Figure 5.1].

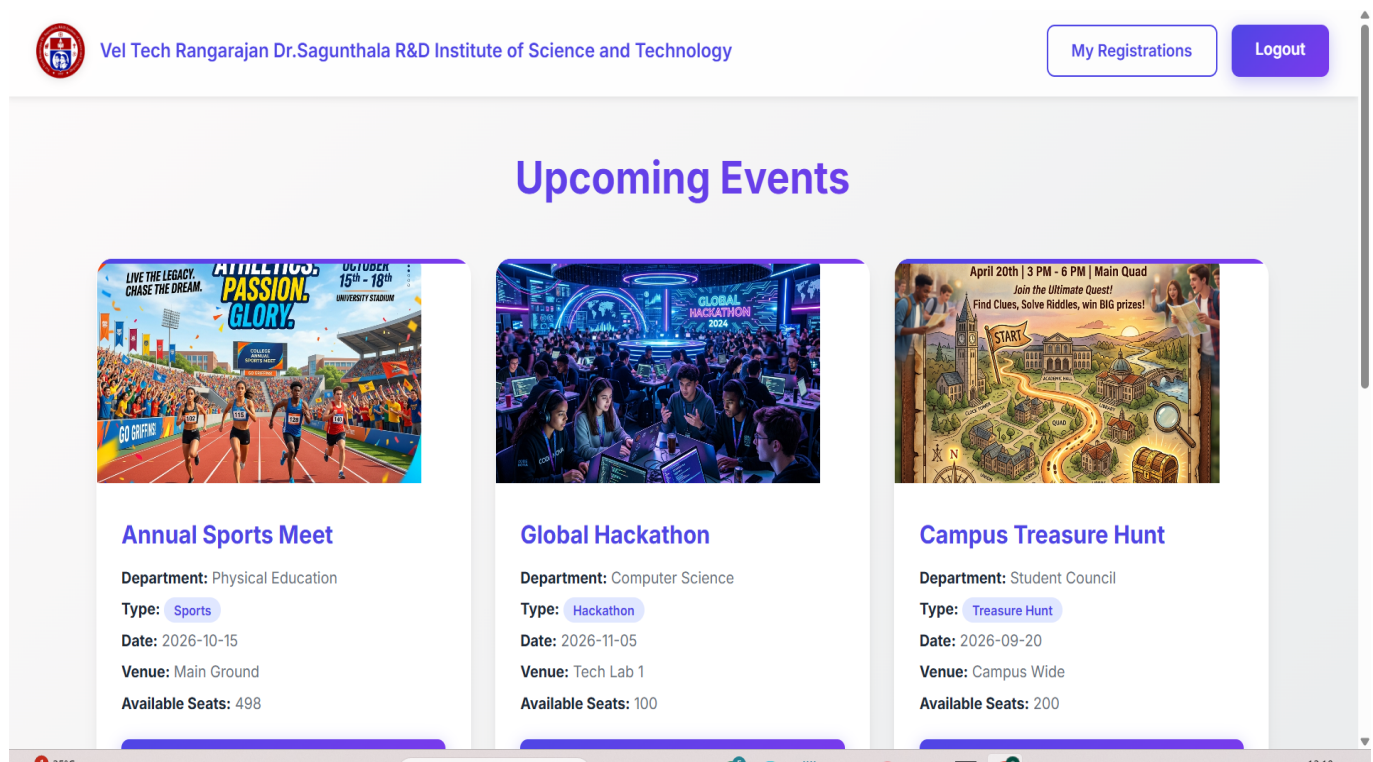
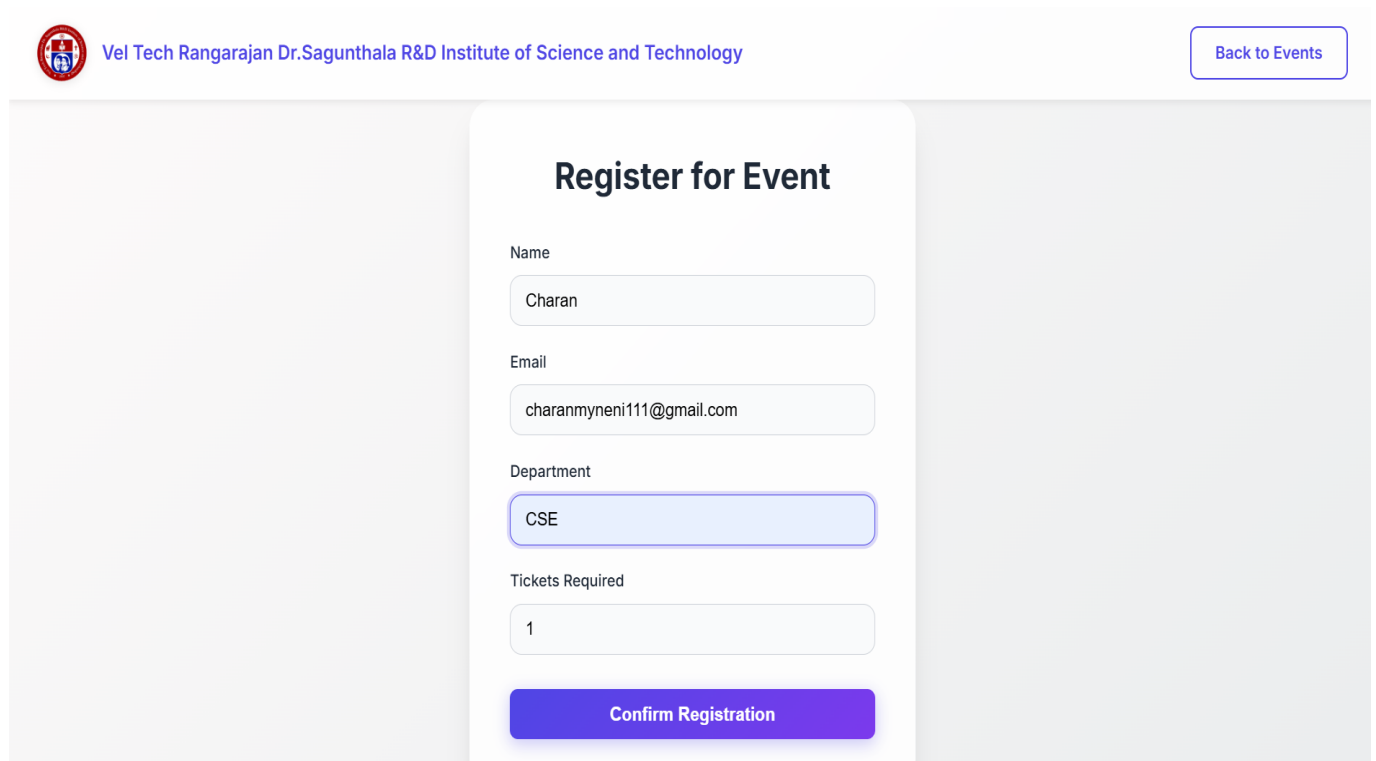


Figure 5.1: Landing Page Interface

The event listing page provides a structured view of all available events within the system. It allows users to browse different categories of events, view event details, and register easily. The interface is designed with clear layouts, filters, and search options to help users quickly find relevant events.

Each event card displays essential information such as event title, date, location,

and description, ensuring users have a clear understanding before registering. The page also supports smooth navigation and responsiveness, enabling users to interact with the system efficiently across different devices[Figure5.2].



Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and Technology

Back to Events

Register for Event

Name
Charan

Email
charanmyneni111@gmail.com

Department
CSE

Tickets Required
1

Confirm Registration

Figure 5.2: Event Registration page

5.3 Discussion

The implementation of the Smart Campus Event Management System demonstrates a significant improvement over traditional event management methods used in educational institutions. This section discusses the integration of various modules, system performance, and overall impact on event coordination.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System successfully achieves its objective of providing a centralized and efficient platform for managing campus events and student participation. The system replaces traditional manual processes with a digital solution that simplifies event creation, registration, and monitoring.

The implementation demonstrates that using a full-stack architecture with a React-based frontend and a Spring Boot backend ensures smooth communication, efficient data handling, and a responsive user experience. Features such as authentication, event management, advanced filtering, and registration validation contribute to a reliable and user-friendly system.

Overall, the project improves operational efficiency, reduces manual errors, and enhances coordination between students and administrators. It serves as a scalable and practical solution for educational institutions to manage events effectively.

6.2 Future Enhancements

While the current implementation of the Smart Campus Event Management System provides a strong foundation for managing campus events and student registrations, the following enhancements are planned for future versions: The system can be further enhanced by introducing advanced features to improve functionality and user experience. Real-time notification systems can be implemented to inform users about event updates, reminders, and registration status using technologies such as WebSockets. Integration of secure payment gateways can support paid events and enable seamless online transactions.

Additionally, a QR code-based attendance system can be introduced to improve participation tracking by enabling faster and more accurate check-ins. An advanced analytics dashboard can provide administrators with detailed insights such as participation trends, popular events, and department-wise performance, helping in better decision-making.

The system can also incorporate automated email and SMS notification services to send alerts for event confirmations, reminders, cancellations, and updates, thereby improving communication and user engagement. Enhancing role-based access control by introducing roles such as Super Admin and Event Organizer can strengthen system security and provide better operational control.

Furthermore, developing a mobile application can significantly improve accessibility and offer a seamless user experience across multiple devices.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The Smart Campus Event Management System addresses a complex engineering problem by integrating frontend, backend, and database technologies to automate campus event management. It replaces manual processes with a scalable digital solution while handling constraints such as user authentication, event capacity, and real-time data processing. The system supports multiple stakeholders including students and administrators, ensuring efficient and reliable operation[Table 7.1].

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK3, WK5, WK6	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The project involves complex engineering activities such as system design, integration of multiple technologies, and implementation of scalable architecture. It demonstrates innovation by transforming traditional event management into a digital platform with real-time updates and efficient workflows[Table 7.2].

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

The Smart Campus Event Management System contributes to sustainable development by promoting digital transformation, improving accessibility to events, and reducing paper-based processes within educational institutions[Table 7.3].

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

Table 7.3: SDG Mapping for Smart Campus Event Management System

Chapter 8

SOURCE CODE

8.1 Source Code

```
1
2 spring.datasource.url=jdbc:h2:mem:campus_events
3 spring.datasource.driver-class=org.h2.Driver
4 spring.datasource.username=sa
5 spring.datasource.password=
6
7 spring.jpa.hibernate.ddl-auto=update
8 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
9
10 spring.h2.console.enabled=true
11 spring.h2.console.path=/h2-console
12
13 spring.mail.host=smtp.gmail.com
14 spring.mail.port=587
15 spring.mail.username=vtu25541@veltech.edu.in
16 spring.mail.password=ixaimknqdumtbtkt
17 spring.mail.properties.mail.smtp.auth=true
18 spring.mail.properties.mail.smtp.starttls.enable=true
19
20 spring.thymeleaf.cache=false
21 server.port=8081
22
23 // index.html
24
```

```

25 <!DOCTYPE html>
26 <html xmlns:th="http://www.thymeleaf.org">
27 <head>
28     <title>Campus Events – Home</title>
29     <meta name="viewport" content="width=device-width,initial-scale=1.0">
30     <link rel="stylesheet" href="/css/style.css">
31 </head>
32 <body>
33
34 <div class="header">
35     <div class="header-brand">
36         
37         <span>Vel Tech Rangarajan Dr.Sagunthala R&D Institute of Science and
Technology </span>
38     </div>
39     <div class="header-actions">
40         <!-- If Admin logged in (we handle it via thymeleaf if session is passed
, or just a generic logout) -->
41         <a href="/admin" class="btn btn-outline" th:if="{session.userRole==
ADMIN}">Admin Dashboard </a>
42         <a href="/my-registrations" class="btn btn-outline" th:if="{session.
userRole!=ADMIN}">My Registrations </a>
43         <a href="/logout" class="btn btn-primary">Logout </a>
44     </div>
45 </div>
46
47
48 <div class="events-grid">
49     <!-- Event Card -->
50     <div class="card" th:each="event:${events}">
51         
52         <div th:if="{event.imageUrl==null}" style="width:100%;height:
200px;background:linear-gradient(135deg,var(--primary),var(--secondary));
border-radius:8px8px000;margin:-25px-25px20px-25px;max-width:calc
(100%+50px);"></div>

```

REFERENCES

Spring Boot Documentation. Available at: <https://spring.io/projects/spring-boot>. Thymeleaf Documentation. Available at: <https://www.thymeleaf.org/>. MySQL Documentation. Available at: <https://dev.mysql.com/doc/>. Hibernate ORM Documentation. Available at: <https://hibernate.org/orm/documentation/>. Eventbrite Platform. Available at: <https://www.eventbrite.com/>. Bootstrap Framework. Available at: <https://getbootstrap.com/>. Java Persistence API (JPA) Documentation. Available at: <https://jakarta.ee/specifications/persistence/>.